

EE-2003

Computer Organization and Assembly Language

Week 04 – Lecture 07

Department of Computer Science/Software Engineering/ Artificial Intelligence
FAST-NUCES Peshawar

September 9, 2025

Outline

- 1 Quick Review
- 2 Memory Addressing
- 3 Code Segment (CS) Register
- 4 Check CS & DS Registers!
- 5 Calculating the Physical Address with Examples

Quick Review: What is Done Last Week?

- 8085 vs 8086 Addressable Memory
- Segmented Memory in 8088
- Data Declaration and Direct Addressing in Assembly
-
- DB vs DW: Memory Layout

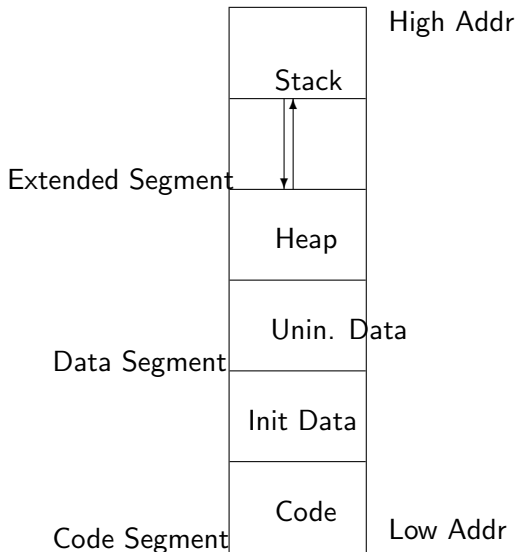
Memory Addressing

- In iAPX 8080/8086, the total addressable space in a 16-bit case is:

$$2^{16} - 1 = 65,535 \text{ (FFFFh)}$$

- Each location stores 1 byte, giving a total of 65,536 bytes = 64 KB.
- 64 KB was insufficient for larger programs as applications grew in size.
- Intel needed to keep backward compatibility so that older 16-bit programs did not need rewriting after production.
- To solve this, Intel introduced segmented memory addressing.
- In this scheme:
 - Code segment at lower addresses.
 - Initialized data segment above code.
 - Heap grows upward.
 - Stack grows downward from higher addresses.
 - All of this resides in RAM.

Memory Addressing



Code Segment (CS) Register

- The CS register contains the base of the code segment.
- Example: CS = 19F5h. The processor appends a 0 $\rightarrow$ 19F50h.
- By shifting left 4 bits, address lines extend from 2^4 to 2^5 .
- This expands addressing capability from 16 bits
($2^{16} - 1 = 65,535 = 0xFFFF = 64 \text{ KB}$)
($2^{20} = 1,048,576 = 0xFFFFF = 1 \text{ MB}$) to 20 bits
- Thus, segmentation allows access to 1 MB memory instead of just 64 KB.

Check CS & DS Registers!

AX 0000	SI 0000	CS 19F5	IP 0100	Stack +0 0000	Flags 7202
BX 0000	DI 0000	DS 19F5		+2 20CD	
CX 0025	BP 0000	ES 19F5	HS 19F5	+4 9FFF	OF DF IF SF ZF AF PF CF
DX 0000	SP FFFE	SS 19F5	FS 19F5	+6 EA00	0 0 1 0 0 0 0 0

CMD >

0100	B80000	MOV	AX,0000
0103	B90300	MOV	CX,0003
0106	BB1D01	MOV	BX,011D
0109	0307	ADD	AX,[BX]
010B	81C30200	ADD	BX,0002
010F	81E90100	SUB	CX,0001
0113	75F4	JNZ	0109
0115	A32301	MOV	[0123],AX

	0	1	2	3	4	5	6	7
DS:0000	CD	20	FF	9F	00	EA	F0	FE
DS:0008	AD	DE	1B	05	C5	06	00	00
DS:0010	18	01	10	01	18	01	92	01
DS:0018	01	01	01	00	02	FF	FF	FF
DS:0020	FF	FF	FF	FF	FF	FF	FF	FF
DS:0028	FF	FF	FF	FF	EB	19	C0	11
DS:0030	A2	01	14	00	18	00	F5	19
DS:0038	FF	FF	FF	FF	00	00	00	00
DS:0040	05	00	00	00	00	00	00	00
DS:0048	00	00	00	00	00	00	00	00

Figure: DOSBox Screenshot

Calculating the Physical Address

- Segment:Offset addressing is used in the 8086/8088.
- Example: Data Segment (DS) = 2FDFh, Offset = 0100h.
- Convert segment to base address:

$$2FDFh \times 10h = 2FDF0h$$

- Add offset to the base:

$$2FDF0h + 0100h = 2FEF0h$$

- Thus, Logical Address 2FDF:0100 = Physical Address 2FEF0h.

Example: Logical to Physical Address Equivalence

- Set $DS = 0900h$, $CS = 0800h$.
- Physical address is calculated as:

$$\text{Physical} = (\text{Segment} \times 10h) + \text{Offset}$$

- For $CS:2000$:

$$0800h \times 10h + 2000h = 0A000h$$

- For $DS:1000$:

$$0900h \times 10h + 1000h = 0A000h$$

- Hence, $CS:2000$ and $DS:1000$ point to the **same physical address**.

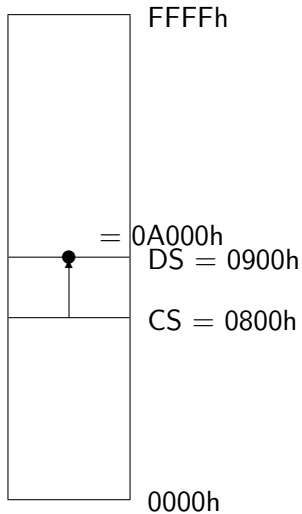


Figure: CS:2000 and DS:1000 both resolve to physical address 0A000h.

Examples of Segment:Offset Overlap in RAM

Segment:Offset	Calculation	Physical Address
CS:2000h:1234h	$(2000h \times 10h) + 1234h$	21234h
DS:2100h:0034h	$(2100h \times 10h) + 0034h$	21234h
CS:3000h:0100h	$(3000h \times 10h) + 0100h$	30100h
DS:3010h:0000h	$(3010h \times 10h) + 0000h$	30100h
CS:1234h:0ABCh	$(1234h \times 10h) + 0ABCh$	12EFCh
DS:12E0h:01FCh	$(12E0h \times 10h) + 01FCh$	12EFCh

Table: Different segment:offset pairs can resolve to the same physical address.

Note: CS and DS registers usually hold different segment values. They do not directly equal each other, but their **segment:offset combinations** can overlap and map to the same physical RAM address.

Set CS & DS Registers with the same values!

```
AX 0000 SI 0000 CS 19F5 IP 0100 Stack +0 0000 Flags 7202
BX 0000 DI 0000 DS 19F5          +2 20CD
CX 001F BP 0000 ES 19F5 HS 19F5   +4 9FFF 0F DF IF SF ZF AF PF CF
DX 0000 SP FFFE SS 19F5 FS 19F5   +6 EA00 0 0 1 0 0 0 0 0

(R) reg=value
CMD >ds=0900

0100 A11701 MOV AX,[0117]
0103 8B1E1901 MOV BX,[0119]
0107 01D8 ADD AX,BX
0109 8B1E1B01 MOV BX,[011B]
010D 01D8 ADD AX,BX
010F A31D01 MOV [011D],AX
0112 B8004C MOV AX,4C00
0115 CD21 INT 21

1 0 1 2 3 4 5 6 7
DS:0000 CD 20 FF 9F 00 EA F0 FE
DS:0008 AD DE 1B 05 C5 06 00 00
DS:0010 18 01 10 01 18 01 92 01
DS:0018 01 01 01 00 02 FF FF FF
DS:0020 FF FF FF FF FF FF FF FF
DS:0028 FF FF FF FF EB 19 C0 11
DS:0030 A2 01 14 00 18 00 F5 19
DS:0038 FF FF FF FF 00 00 00 00
DS:0040 05 00 00 00 00 00 00 00
DS:0048 00 00 00 00 00 00 00 00

2 0 1 2 3 4 5 6 7 8 9 A B C D E F
DS:0000 CD 20 FF 9F 00 EA F0 FE AD DE 1B 05 C5 06 00 00 = f.n= i|.+....
DS:0010 18 01 10 01 18 01 92 01 01 01 01 00 02 FF FF FF .....f. ....
DS:0020 FF FF FF FF FF FF FF FF FF FF FF FF EB 19 C0 11 .....δ.L.
DS:0030 A2 01 14 00 18 00 F5 19 FF FF FF FF 00 00 00 00 6.....J. ....
DS:0040 05 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....

1 Step 2ProcStep 3Retrieve 4Help ON 5BRK Menu 6 7 up 8 dn 9 le 10 ri
```

Figure: DOSBox Screenshot

Check M1 & M2 Window to verify the results!

AX 0000	SI 0000	CS 0800	IP 0100	Stack +0 0000	Flags 7202
BX 0000	DI 0000	DS 0900		+2 20CD	
CX 001F	BP 0000	ES 19F5	HS 19F5	+4 9FFF	OF DF IF SF ZF AF PF CF
DX 0000	SP FFFE	SS 19F5	FS 19F5	+6 EA00	0 0 1 0 0 0 0 0

CMD >				1	0	1	2	3	4	5	6	7
				CS:2000	02	B2	08	B6	08	E8	07	07
				CS:2008	72	21	80	3E	6E	03	01	74
				CS:2010	09	8B	36	78	03	B0	B2	F9
				CS:2018	EB	11	A1	6B	03	89	05	A0
				CS:2020	71	03	88	45	02	C6	06	5D
				CS:2028	03	04	F8	C3	83	C3	02	B0
				CS:2030	02	B2	33	B6	3B	E8	D7	06
				CS:2038	72	27	E8	FB	0D	72	1B	E8
				CS:2040	9E	0D	73	1D	E8	DA	0C	72
				CS:2048	18	A0	5E	03	24	FE	3C	86

2	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	
DS:1000	02	B2	08	B6	08	E8	07	07	72	21	80	3E	6E	03	01	74	 r!< >n..t
DS:1010	09	8B	36	78	03	B0	B2	F9	EB	11	A1	6B	03	89	05	A0	.i6x. . ik.e.A
DS:1020	71	03	88	45	02	C6	06	5D	03	04	F8	C3	83	C3	02	B0	q.eE. .l ..° ā .
DS:1030	02	B2	33	B6	3B	E8	D7	06	72	27	E8	FB	0D	72	1B	E8	. 3 : . r' dJ.r.δ
DS:1040	9E	0D	73	1D	E8	DA	0C	72	18	A0	5E	03	24	FE	3C	86	R.s.δr.r .ā^.\$#<ā

1 Step	2ProcStep	3Retrieve	4Help ON	5BRK Menu	6	7 up	8 dn	9 le	10 ri
--------	-----------	-----------	----------	-----------	---	------	------	------	-------

Figure: DOSBox Screenshot

Why Segmentation and Relationship of CS and DS

- Segmentation allows the 8086/8088 to access **1 MB of memory** using only 16-bit registers.
- A physical address is calculated as:

$$\text{Physical Address} = (\text{Segment} \times 10h) + \text{Offset}$$

- The Code Segment (CS) register holds the base of the current program instructions.
- The Data Segment (DS) register holds the base of the program's data.
- Both CS and DS can produce the **same physical address** if their segment:offset pairs resolve equally.
- This provides **backward compatibility** (16-bit programs) while enabling access to a larger memory space.

Thank You!

Any Questions?

- Today we explored: **segmentation, logical vs physical addresses, CS/DS relationship.**
- Key lesson: **Different segment:offset pairs can overlap to the same RAM address.**
- Segmentation extends 16-bit registers to access **1 MB memory space.**

"If you understand memory addressing, you understand the CPU."